

SOLANA VALUATION INTERFACE

Pricing that anyone can recompute.

The value of a protocol asset, computed on-chain from verified accounts and written to a public account — **so nobody has to trust an API doing hidden math.**

A design-partner proposal for Hyla

Prince · SVI · September 2026

“

honestly an accepted smart contract interface for pricing onchain assets would be amazing — bc pyth wants us to read our own protocol by RPC and give them data by an API — it leaves all sorts of unimaginable security holes

© Plish | Hylo · 28 August 2026

You identified it, unprompted.

This deck is the thing you described, built — with Hylo as the first adapter.

You were right about the risk.

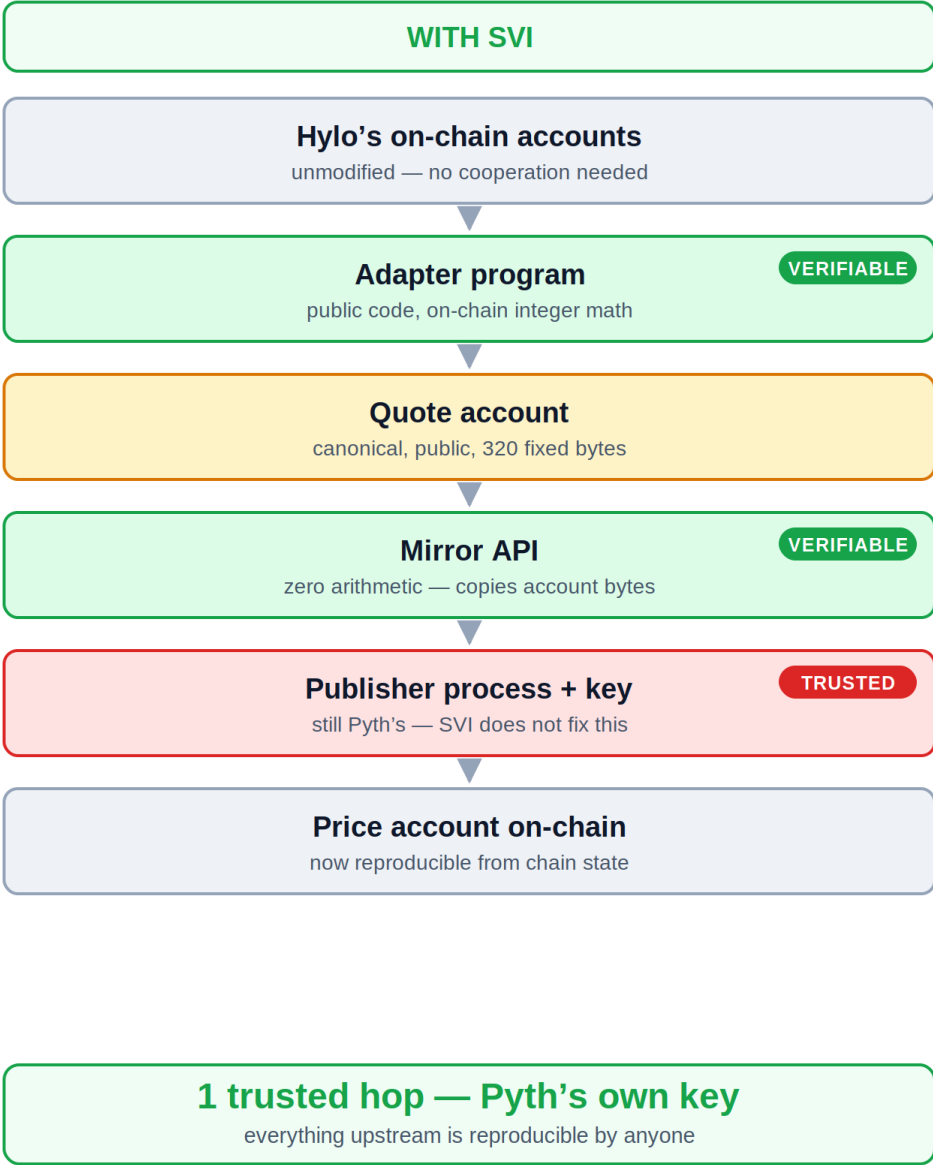
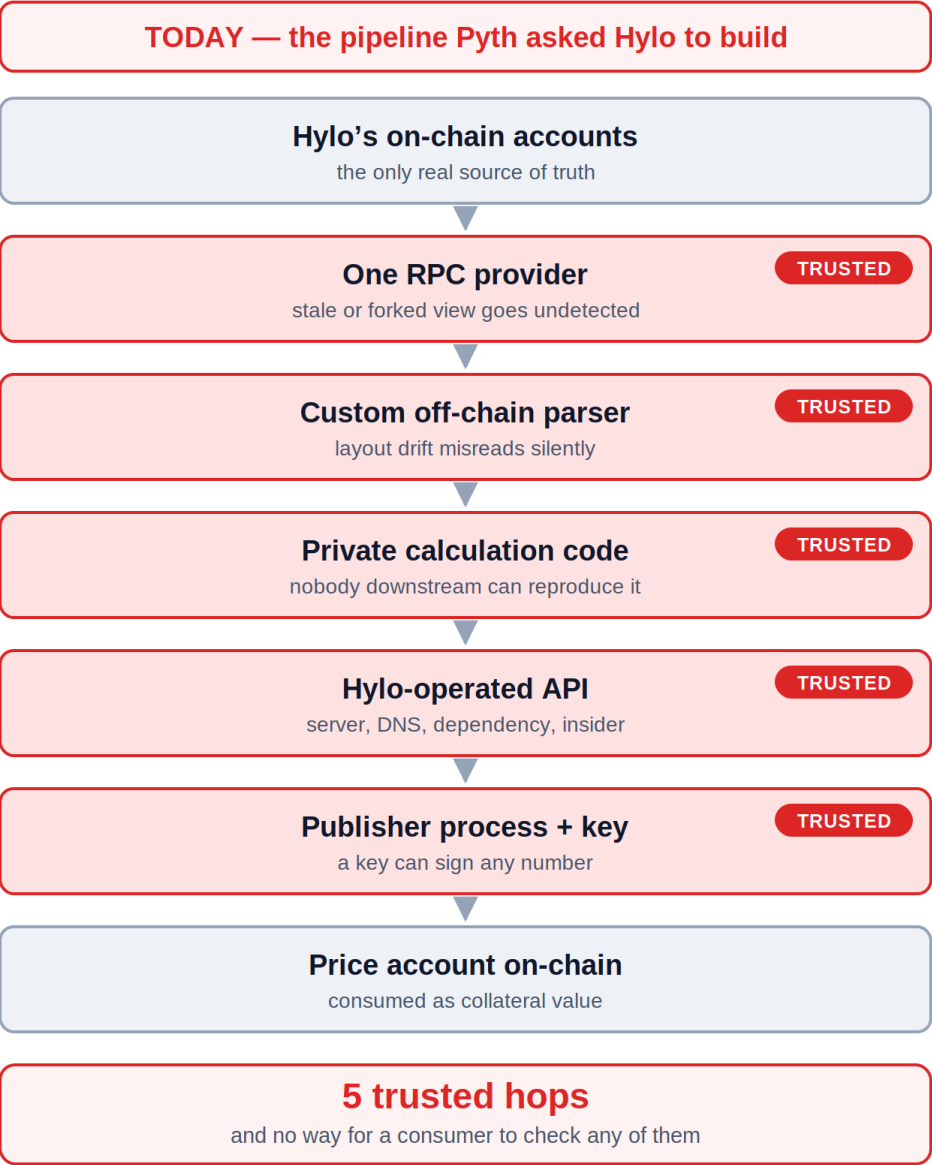
That pipeline has five trusted hops and no way for a consumer to check any of them.

It is not a hypothetical.

Four lenders have already lost money to exactly this class of failure.

Where does the number come from?

Five places the value can change without anyone noticing — versus one, whose output anyone can reproduce from the same slot’s account data.



This failure class already has a body count

In every case below, no on-chain account was ever wrong. The loss came entirely from the valuation layer between true state and the consumer.

\$5.8M

Loopscale

Apr 2025 · Solana

The lender's own pricing of RateX PT tokens was wrong. Undercollateralised loans drained 12% of TVL — two weeks after launch.

\$1.8M

Moonwell

2025 · bad debt

A config change priced cbETH via cbETH/ETH only, dropping the ETH/USD leg. It read \$1.12 instead of ~\$2,200. Bots did the rest.

1.06 → 1.7

wUSDM / Venus

Feb 2026

A donation attack moved the ERC-4626 share exchange rate itself, then self-liquidated against it. The rate was the attack surface.

\$10.2M

YieldBlox

Feb 2026

Thin-liquidity VWAP oracle on a tokenised-treasury pair inflated collateral value enough to over-borrow.

\$137M+

lost across DeFi exploits in Q1 2026 alone; over \$1B cumulatively in 2026.

Loopscale currently lists hyUSD. A lender that hand-rolls valuation for every protocol asset it accepts will eventually hand-roll one wrong.

THE IDEA

Move the arithmetic on-chain.

Then RPCs, APIs and keys become couriers —
not authors — of the number.

Computed

by a Solana program, from accounts it verifies
itself

Published

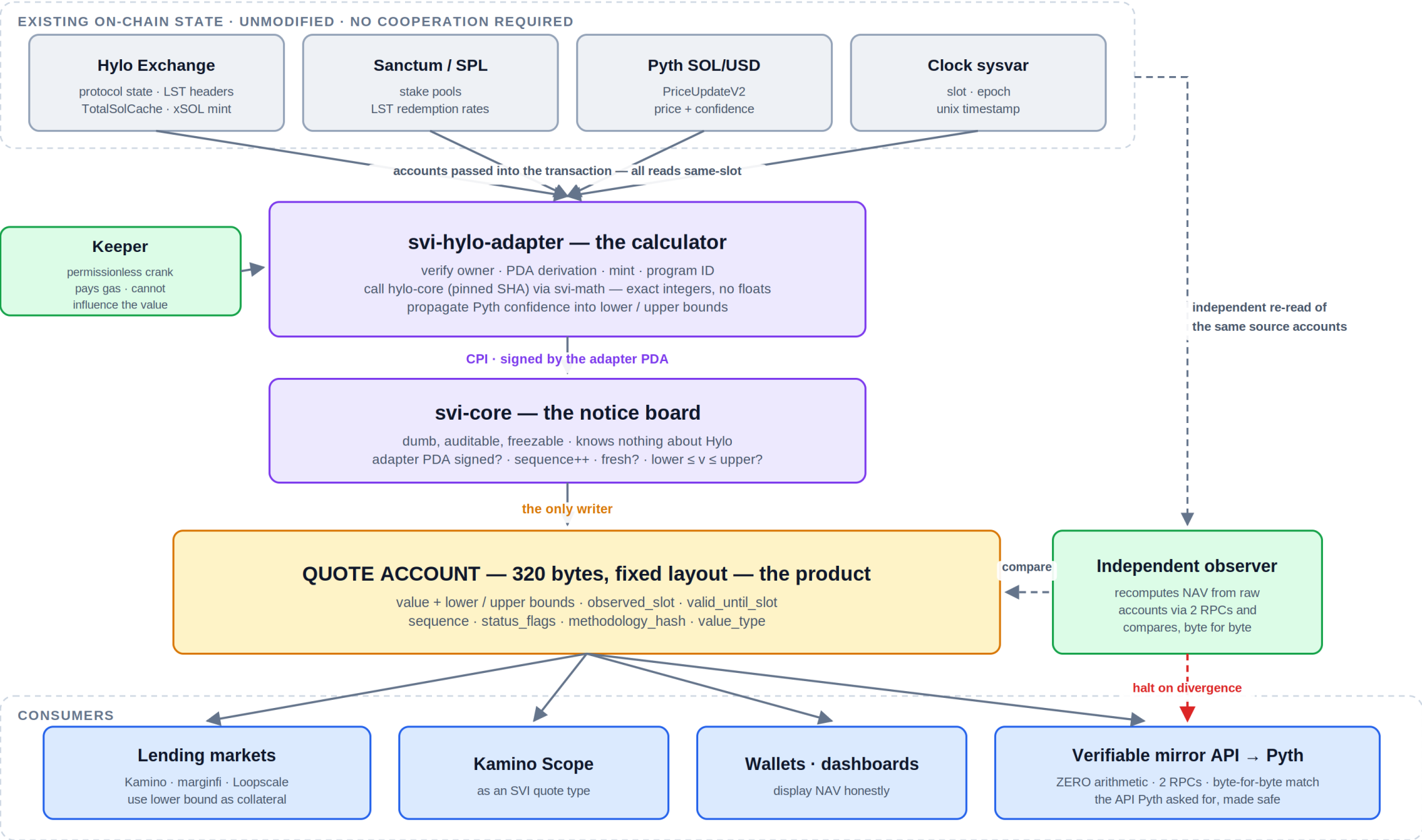
to a canonical 320-byte account anyone can
read

Reproducible

byte-for-byte by any third party, from the same
slot

Solana Valuation Interface — system architecture

The value is computed on-chain from verified accounts and written to a public account. No off-chain service is ever trusted with the arithmetic.



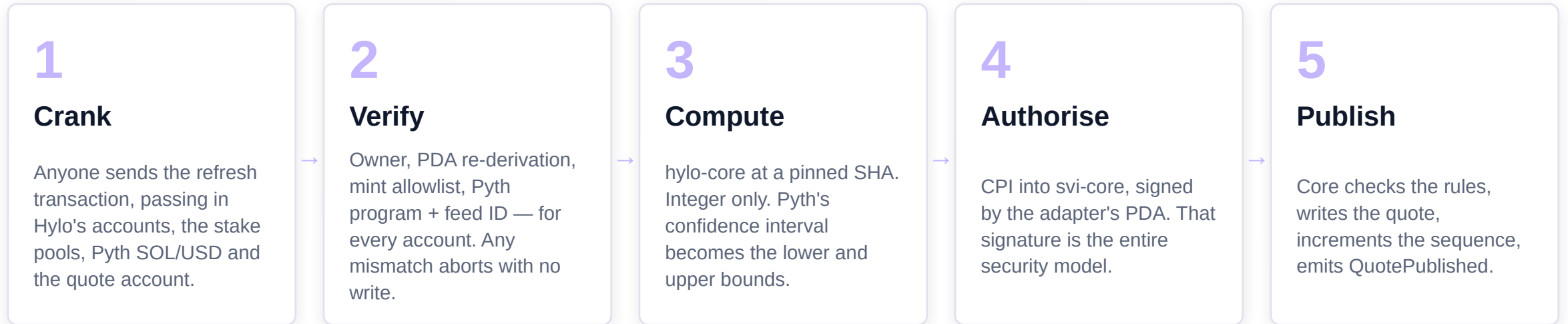
Fail stale, never fail wrong. Any unverified input aborts the refresh — the quote ages out and consumers see the expiry. No code path publishes a value computed from unvalidated inputs.

Five pieces, and what each one is for

- | | | |
|---|--|---|
| 1 | Adapter <i>the calculator</i> | One per protocol. Reads the protocol's public account bytes, verifies owner / PDA / mint / program ID, and calls the protocol's own math library. Reimplements nothing. |
| 2 | Core <i>the notice board</i> | Tiny, protocol-agnostic, auditable — and one day freezable. Only a registered adapter's PDA may write. Sequence monotonic, freshness enforced, bounds checked. |
| 3 | Quote account <i>the product</i> | 320 fixed bytes: value, lower/upper bounds, observed_slot, valid_until_slot, sequence, status_flags, methodology_hash, value_type. |
| 4 | Keeper <i>rings the bell</i> | Permissionless — anyone can crank. Pays gas; cannot influence the value. If nobody cranks, the quote expires visibly. It fails loud; it never lies. |
| 5 | Observer <i>the second accountant</i> | Recomputes from raw accounts through independent RPCs. A divergence of even one unit alerts and halts the mirror API. |

The principle it all reduces to: there is no point in the pipeline where a human or a hackable server chooses the number.

One transaction, all reads from the same slot



Permissionless, yet uninfluenceable

Anyone can send this transaction. Nobody can change what it writes — the value is computed by adapter code from verified accounts, and only the adapter's PDA can authorise the write. The keeper is paying for gas. If our bot dies, anyone else's bot cranks it.

Epoch boundaries, handled

When TotalSolCache is an epoch behind, the refresh bundles Hylo's own update_1st_prices crank — permissionless, payer-only signer — then re-reads and proceeds, flagging EPOCH_BOUNDARY_CPI. If that CPI won't fit, we abort rather than publish a stale NAV.

THE DESIGN RULE

Fail stale, never fail wrong.

No code path may publish a value computed from unvalidated inputs. A visibly-expired quote is always preferable to a plausible wrong one.

Condition	Behaviour	What the consumer sees
Everything healthy	Publish, sequence++	Fresh quote, flags clear
New epoch, LST crank not run	Bundle Hylo's permissionless crank, then publish	EPOCH_BOUNDARY_CPI set
...and that CPI won't fit	Abort — no write	Quote ages past valid_until_slot
CR < 100% (Destabilized)	Publish 0 with halt flags	DESTABILIZED — collateral value zero, not a dip to buy
Pyth stale / wrong feed / out of Hylo's own tolerance	Abort — no write	Stale quote, expiry visible
xSOL supply == 0	NAV = exactly 1.000000000	ZERO_SUPPLY_DEFAULT
Target program upgraded (layout drift)	Abort until reviewed	Stale quote, expiry visible
Observer disagrees with chain	Alert + halt the mirror API	Mirror stops; the account is still readable

SVI imposes no oracle tolerance of its own — it reads Hylo's on-chain OracleConfig fresh on every refresh, so the feed reflects exactly what the protocol itself would accept for a real redemption.

Three ways to consume it, one of them dominant

A Read the note 99% of usage

- Pass the quote account into your own instruction
- Check `valid_until_slot` and `status_flags`
- Check `value_type` is what you onboarded
- Use the lower bound for collateral

One account read. Fixed 320-byte layout, asserted at compile time.

B Call the calculator live high-stakes only

- Include the adapter's compute in your own transaction
- Value derived from this exact slot's state, atomically
- More accounts, more compute units
- Zero staleness

For liquidators who cannot accept a 60-second-old quote. Most integrators never need this.

C The verified mirror off-chain consumers

- Fetch the account through two independent RPCs
- Require byte-for-byte agreement
- Decode and serve JSON with full provenance
- Serve an error when stale — never a last-known-good value

A photocopier. It cannot lie about the price, because anyone can compare its output to the account.

Compute-budget reality: an xSOL NAV computation costs tens of thousands of CU. That is fine in a dedicated keeper transaction and often not fine inside a liquidation. Mode A is the default read path, which is why the stored quote account — not the atomic preview — is the product.

Zero changes to your programs. Ever.

Solana programs have no view functions

— and none are needed. Account data is public bytes. The adapter is handed your accounts as transaction inputs and deserializes them itself, verifying owner, PDA derivation, mint and program ID before touching a single field. All reads inside one transaction are same-slot consistent by construction.

The one CPI needs no permission either

The only thing SVI calls into Hylo is `update_lst_prices`, at epoch boundaries, to accrue state that is stale until cranked. It takes a payer-only signer — it is already permissionless. So even the single CPI in the design requires nothing from you.

NOT ASKED FOR

- A program upgrade
- An engineering sprint
- Integration work
- Infrastructure to run
- Funding
- Exclusivity

THE ENTIRE ASK

- **Five technical answers**
- **Permission to name Hylo as design partner**
- **A co-announcement at launch**

Roughly one engineer-afternoon in total.

We reimplement none of your math

The adapter depends on hylo-core, pinned to an exact git SHA recorded on-chain in the adapter config. Your redemption math and SVI's published value cannot diverge, because they are the same code.

Quantity	hylo-core function	Rounding, taken from your source	SVI field
xSOL NAV, redeem side	exchange_math::next_levercoin_redeem_nav	mul_div_floor × price.lower	quote_amount and lower_quote_amount
xSOL NAV, mint side	exchange_math::next_levercoin_mint_nav	mul_div_ceil × price.upper	upper_quote_amount
Collateral ratio	exchange_math::collateral_ratio	mul_div_floor, price.lower	status_flags zone bits
hyUSD NAV if depegged	exchange_math::depeg_stablecoin_nav	mul_div_floor	hylo-hyusd-backing-v1

Why the headline number is the redeem side

Hylo already prices NAV as a range and always takes the side favourable to the protocol — mint uses ceil math with Pyth's upper bound, redeem uses floor with the lower. A holder liquidating xSOL realises the redeem side, so redeem NAV is the defensible collateral value. Publishing the full range preserves the information for consumers with other needs. That is your convention, not ours.

Fixed-point domain

UFix64<N9> for NAV and prices, UFix64<N6> for supplies — per hylo-core convention.

No floating point anywhere in SVI, by design. u128 intermediates, every function total.

This is not a pitch for something that might exist

Component	State	What it is
crates/svi-math	DONE	Exact integer math. u128 intermediates, mandatory explicit rounding, no_std, arithmetic side effects denied, every function total, property-tested.
programs/svi-core	DONE	Descriptor + 320-byte zero-copy Quote, layout asserted at compile time. publish_quote enforces: feed active, adapter PDA signed, bounds coherent, observation fresh and strictly newer.
programs/svi-mock-adapter	DONE	Exercises the CPI and authority path end to end.
hylo-xsol-nav-v1 spec	DRAFT v0.9	Accounts, formulas, rounding, edge cases, failure states, output mapping, update policy, verification procedure.
programs/svi-hylo-adapter	BLOCKED	Scaffolded. Waiting on five answers — not on engineering time.
Keeper · mirror API · observer	PHASE 2	Built once the methodology is frozen.

The critical path is not engineering capacity. It is five answers from Hylo.

What Hylo gets for an afternoon



Zero engineering cost

No program changes, no integration, no infrastructure to run or be paged for.



The Pyth request answered

Pyth gets exactly the API shape they asked for — minus the security objection you raised.



Liability moved off your team

The valuation code is public, reproducible, and not yours to operate or be blamed for.



A feed lenders can underwrite

Conservative redemption-anchored value, explicit bounds, freshness stamp, and a DESTABILIZED flag that unambiguously means zero.



V2 scales down this path

The xAsset Engine turns 4 feeds into $4 \times N$. A methodology-per-asset framework absorbs that; a hand-rolled API does not.



A public artifact for Hylo

hylo-xsol-nav-v1 is a rigorous public specification of Hylo's valuation semantics — useful documentation in its own right.

The asset class is NAV-shaped, and it is growing fast

\$3.7B

tokenised RWA on Solana,
up 4× in six months

~\$15B

Solana LST supply —
every unit a formula, not a
trade

~\$3.5B

lending TVL carrying this
valuation risk today

2,800+

Pyth feeds — built for
observed market prices

Why now

- A design partner has the problem today, with a deadline — Pyth has already asked.
- Hylo V2's xAsset Engine multiplies 4 feeds into $4 \times N$.
- Solana RWA quadrupled in six months, and it is NAV-priced by definition.
- The Foundation is standardising the neighbouring layer (sRFC 40) right now.
- Colosseum runs 28 Sep – 2 Nov 2026.

The honest version

Standards do not monetise. Realistic year-one revenue is low six figures at best, from services beside a permanently free standard — custom adapters, managed keeper infrastructure, reviews. There is no token and there will not be one.

The return here is position, not revenue: authoring the valuation interface Solana adopts, with live feeds and a named partner, while the Foundation is standardising next door. Cost to find out: under \$500 to deploy and \$50–150 a month to run.

Nobody does this as an open interface

Player	What it is	Why SVI is not redundant
Pyth	Push/pull oracle, 2,800+ feeds, 128 publishers	Customer, not competitor. Built for observed market prices — it has no mechanism for reproducible protocol NAV, which is why it is asking Hylo for an API.
Switchboard	Permissionless custom feeds, TEE-executed jobs	Real overlap, but the math still runs off-chain in an enclave. You trade 'trust the API' for 'trust the enclave and the job definition'.
Kamino Scope	Oracle aggregator, ~40 asset-specific valuation types	Closest analogue and proof the adapter model works — but it is Kamino's internal component, curated by and for Kamino. A consumer target, not a rival.
Blueshift Doppler	Ultra-efficient price publishing primitive	Transport, not semantics. It makes writing a number cheap; it says nothing about where the number came from.
sRFC 40	Foundation + Exo vault standard, NAV-settled flows	The one genuine strategic risk. Response is to contribute into it, not compete — and to cover what vaults don't: leveraged tokens, backing NAV, redemption, receipt tokens.
Chainlink NAV	Brings off-chain fund NAV on-chain for RWAs	Opposite direction: it transports NAV computed off-chain by a fund administrator. SVI computes NAV from on-chain state.

The honest read: there is no one doing exactly this, and that is not because it is hard — it is because it does not obviously monetise. Kamino, Switchboard and Blueshift could each build it in weeks. The defensible assets are methodology specs, audits, track record and partner relationships — not code.

Six stages, each with a gate before the next

0	Spec, co-signed	Hylo answers Q1–Q5. Methodology frozen and hashed.	NOW — the entire ask
1	Devnet	Adapter live. Differential test vs hylo-core over 1,000+ randomised states.	~3 weeks
2	Mainnet + mirror	4 feeds live. Mirror serves Pyth's schema and does zero arithmetic.	~6 weeks
3	First consumer	One protocol reads a quote account in production.	Q4 26 – Q1 27
4	Adapter #2	A second protocol ships with zero changes to core or math.	Q1 2027
5	Into the sRFC	Submitted aligned with sRFC 40, not parallel to it.	Q1–Q2 2027

Stages 0–2 are within our control and only prove competence. Stages 3 and 4 are the real test — until a second protocol adopts it, SVI is a Hylo wrapper, and we say so.

THE ASK

Five answers, a name, and a retweet

Q1 V1 vs V2 layout, and timing

The equations doc describes V2, the addresses page lists V1 programs, and the SDK has cbBTC / exo machinery. Which layout is live on mainnet today, and is any account structure changing before November?

Q2 update_1st_prices via CPI

Any constraint preventing invocation from a third-party program — LUT requirements, remaining-accounts size? This decides whether the epoch-boundary path bundles your crank or degrades for a window.

Q3 Pyth's exact request

Which feeds, what cadence, what response schema? Highest-value answer of the five: it defines the mirror contract. A forwarded message is enough.

Q4 SDK release pinning

Will Hylo tag SDK releases aligned to program deployments, so the adapter can pin hylo-core to the deployed program version?

Q5 Design-partner blessing

May SVI name Hylo as design partner for the spec and the Colosseum submission, and would Hylo co-announce at launch?

Deadline that matters: Colosseum runs 28 Sep – 2 Nov 2026. Answers by mid-September make a mainnet-live submission with Hylo named as design partner realistic.

Six weeks from spec freeze to Pyth evaluating

Phase	Duration	What ships	Hylo's involvement
0 · Spec freeze	1 week	Q1–Q5 folded in; methodology hashed and frozen at v1.0	One review pass
1 · Devnet	2 weeks	Adapter computes xSOL NAV; differential test vs hylo-core; failure paths tested	None
2 · Observer parity	1 week	Zero divergence over 72h continuous; compute budget under 200k CU	None
3 · Mainnet feeds	1 week	xsol-nav, hyusd-backing, ehyusd-rate, xsol-leverage live; keeper up	Optional: co-run a keeper
4 · Mirror + Pyth	1 week	Mirror API in Pyth's schema, zero arithmetic; conformance suite public	An intro to Pyth, if willing
5 · Co-announce	—	Public launch	A retweet

Hylo's cumulative time across all six phases is measured in hours.

Exit conditions: if Hylo wants the feeds retired we deprecate them publicly and immediately; the adapter is open source and yours to fork or take over; nothing here creates exclusivity in either direction.

The objections worth raising, answered

“This makes SVI a dependency for our asset pricing.”

It doesn't, and that is deliberate. Cranking is permissionless — Hylo can run a keeper. The adapter is open source, the quote account is readable with no SVI infrastructure involved, and the methodology is a public document. If SVI disappears tomorrow, everything except the mirror keeps working — and the mirror is the piece that does no math.

“What if your adapter is wrong and someone gets liquidated?”

A real risk. Hence: call hylo-core rather than reimplement it; an independent observer that halts on a one-unit divergence; abort rather than publish on any unverified input; and no real-collateral use until a third-party audit. Until that audit, our own position is that no lending market should underwrite against these feeds.

“Is this a hackathon project that dies in November?”

Partly — that is what the funding path is for. Mitigations that hold regardless: everything open source, the keeper permissionless, the core small enough to freeze, and the methodology spec standing on its own as documentation of Hylo's valuation semantics even if every line of SVI code were abandoned.

“What's in it for you?”

Reputation, a Colosseum submission with a real design partner, and a shot at authoring a Solana standard. Not revenue — the standard and SDKs are permanently free and there is no token. Worth being blunt about, because an unexplained motivation is its own kind of risk.

WHAT HAPPENS NEXT

- 1 You** Answer Q1–Q5. Telegram, a call, or comments on the spec — whichever is least effort.
- 2 Us** Fold them in, freeze hylo-xsol-nav-v1 at v1.0, publish the hash.
- 3 Us** Build, test, ship to devnet, run the observer to parity.
- 4 You** One review pass on the frozen spec, and a yes or no on being named.
- 5 Us** Mainnet feeds, mirror API, Pyth intro.
- 6 Both** Co-announce.

The methodology spec is complete except for those five answers. Reading §§2–6 takes about fifteen minutes and is the fastest way to check whether we have understood Hylo's math correctly — which, if we haven't, is the most valuable thing you could tell us.

Fail stale, never fail wrong.

github.com/princdotrs/solana-valuation-interface